

Ryo Hunte Data sharing system

Introduction and requirements analysis:

System Overview:

The project requires the building of a secure local interprocess communication system using POSIX C. The system will allow local clients to share, access and update data through a background daemon process.

The program consists of 3 main components:

- Daemon : In charge of the management of data, manages metadata and enforce access permissions
- C Library : Provides access to the daemon functionality
- Client programs : Programs that use the library to perform operations

Functional requirements:

To ensure that the program is workable and secure, the following functional requirements and security measures are essential.

Daemon FR's	FR-1 - Run constantly in the background FR-2 - Store and retrieve data blocks in memory FR-3 - Read the data length and accompanying metadata FR-4 - Enforce access controls based on predefined permissions
C library FR's	FR-1 - Create function sendNewBlock() to transmit a new data block with metadata to the daemon FR-2 - Create function getBlock() that retrieves an already stored block
Client FR's	FR-1 - Link against the C library to access daemon functionality FR-2 - Perform actions based on permission rights

To follow the CIA framework, these security features will be implemented:

Security feature	Approach/Execution
SF-1 - The daemon will enforce role based access controls	The daemon will maintain access control per block (read/write). Before processing any requests, the daemon checks the UID and matches it against the permissions before starting operations
SF-2 - Authenticate clients using ID	Enforced through file system permissions
SF-3 - Use unix domain sockets stored in secure directories	Will be stored in tmp/daemon.sock and will apply umask() to prevent unauthorised access
SF-4 - Implementation of secure memory management	Use of memset() and strncpy() functions

SF-5 - Implementation of bounded buffers to prevent DoS attacks	All data from clients will be stored in fixed size buffers using a strict length to prevent memory corruption
SF-6 - Log events that occur such as creation, access and modifications	Creation of logging system that records events with information of UID

Design, analysis and implementation:

IPC Methods:

To determine the best IPC method approach to the system, I carried out some research on the methods I could potentially implement for my program:

- Pipes : A natively supported Linux IPC mechanism and allows for simple unidirectional communication between child and parent processes. Whilst they are easy to implement, pipes pose some security issues as they do not support access control or authentication, both of which are essential for a secure data sharing system. In addition to this, they are limited to related processes which may not be suitable for a daemon that interacts with independent clients
- Shared memory: A high speed communication mechanism allowing multiple processes to access the same area of memory (more suited for high throughput systems). Despite its fast communication, the building of this IPC requires more mechanisms for explicit synchronisation which adds a lot of complexity to the program.
- Signals: An IPC mechanism used to notify processes of events. This mechanism is not suitable for the design of the system as it does not support data transfer, only the notification of events and therefore cannot handle the complexity of the intended program.
- Unix domain socket : Allows for full-duplex datagram based communication allowing unrelated processes to communicate (clients and the daemon) and supports bidirectional, structured communication. Whilst it poses a lot of advantages, it is local only communication and does not allow for network communication. With large amounts of data, it can be slower than other IPC's as it avoids kernel copies leading to low latency.

After carefully evaluating the options, Unix domain socket seems to be the most appropriate approach to the system. Despite it having some drawbacks, it is ideal for request-response models and more secure as it is file based which enables access control using unix permissions. It also provides easy integration with POSIX system calls such as read() and write().

Storage architecture:

As mentioned above, the system will have a structured approach to store data by creating a block-like structure encapsulating all of the relevant metadata and content. Each variable/ piece of data is an instance of a block type allowing the daemon to manage, store and retrieve data securely. The blocks will be managed by an array of pointers and a counter will be used to track the number of active blocks. The block structure will include:

- ID - A unique identifier for the block
- Secret - A 16 byte array
- Length_data - The actual size of the data content
- Data - A generic pointer to the stored data
- Permission - A byte represent the access rights of the block

System design:

Server main components :

The main components of the daemon include :

The daemon will run continuously in the background listening on the UNIX domain socket for any client requests. The daemon will then process requests and perform checks before any necessary operations on the data.

1. Handling client connections through a UNIX domain socket
2. Managing data storage using the Block method mentioned above
3. Responding to clients on whether the process was a success or failure
4. Role based access control

Implementation:

Socket creates using `socket(AF_UNIX, SOCK_STREAM, 0)` with the socket having a secure path using `bind()`. Communication with the client is handled through `read()` and `write()` calls. Socket cleanup is also handled using `unlink()` which removes the socket file.

Each data unit stored in the Block struct is allocated dynamically. Each new block is stored with the `malloc()` function. The method `strncpy()` is used to securely copy the ID. Data is a `*void` pointer to allow arbitrary data to be stored. The blocks are stored in an array `Block *blocks[MAX_BLOCKS]` and is indexed with `blocks_count`.

Secure coding practices such as the utilisation of `memset()` are applied before freeing sensitive data needed for data such as secrets.

C Library:

The C library will act as the middleman allowing the clients to interact with the daemon through the functions in the program. The library provides a simple interface for clients whilst abstracting low level socket communication and data manipulation.

1. Establish socket connection to the server
2. Creating and formatting requests
3. Receiving responses from the server

How the library talks to clients:

1. `sendNewBlock()` - This function will create new data blocks and send them to the daemon
2. `getBlock()` - this functions allows clients to retrieve data based on access permissions

Clients:

The clients use the library to interact with the server.

1. Call the C library's API
2. Provide the necessary input such using the data from the Block
3. Handle success or failure messages
4. Present to the user the messages

Component communication:

A small diagram demonstrating the flow of the program:

Client program → Calls function from the **C library** → **C Library** constructs a request and send it to the daemon through the Unix socket



Daemon:

- Firstly, it parses the request

- Authenticates the secret
- Checks against the permissions
- Performs operation based on access control
- Sends back through the socket



C library receives a response and returns to → **client program**

Design of the API :

In this program, the C library is the API. The purpose of this is to get rid of all complexities handled and provide a simple and secure program for the clients to use. There are 2 core operations that the API handles:

Operation	Purpose
sendNewBlock()	Allowing clients to send new block of data
getBlock()	Retrieves existing block of data if authorised

The API focuses on simplicity using standard data types such as (char*, uint8_t*, uint32_t*) to continue consistency with POSIX C and avoid complex objects. This makes it easier to integrate into low level C programs. All functions are declared in a .h file named library.h. Developers comments are included in the program so that the purpose is clarified. For example:

```
uint8_t sendNewBlock(char *id, uint8_t *secret, uint32_t data_length, void *data);
```

```
//id = unique string identifier for block
//secret = 16 byte authentication string
//data_length = length of data in bytes
//data = pointer to blocks data
```

Portability:

Porting to/from other API's or IPC's would be relatively easy due the simplicity and abstraction of the existing design. The API currently exposes a small number of well defined functions whilst encapsulating all lower logic within the library itself. As a result, porting to a different IPC mechanism (such as message queues, TCP sockets, shared memory) would not require any changes to client code. The client interface and logic could remain the same whilst only a minor change in implementation within the library itself. The API is also future proof as it supports the addition of more functions.

Porting to an existing application that uses other shared data (such as database systems and REST API's), would be more challenging. This is due to the difference in languages, as this system is written in C it will be less accessible to developers working with Python and Java. In addition, many systems abstract away authentication and memory handling; however, this system requires explicit handling of permissions, secrets and buffer memory.

Testing:

Test cases:

TC-1: Daemon is running:

Purpose : Ensure the daemon can be executed without crashing

Input : Terminal commands such as `./daemon & ps aux | grep daemon` to check if the daemon is running

Expected output: Daemon running on terminal with no errors

TC-2 : Library is running

Purpose: Ensure the library compiles and integrates with the client

Input: Compiling library with client

Expected output : No compiler errors or crashes

TC-3: Accepting client connection

Purpose: Ensuring the daemon accepts client connection through the unix domain socket

Input: Client attempts to connect via the library - terminal command `./client`

Expected output: connection accepted without error

TC-4: Block is sent

Purpose: To verify `sendBlock()` function is working

Input: ID: client1 secret = 0

Expected output: return 0 and message : block stored

TC-5 : Block is retrieved

Purpose: to verify `getBlock()` function is working

Input: ID: client1 secret = 0

Expected output: The data of the previous process has been returned

TC-6: Invalid client

Purpose : To ensure that the process fails if the secret is longer than 16

Input: char = {99, 99, 99, 99,99,99, 99, 99, 99,99, 99, 99, 99, 99, 99 99, 99, 99, 99,99}

Expected output: Error in storing the block

TC-7: Reading permissions

Test case responses:

TC-1 response:

```
ryohunte@Ryos-MacBook-Pro Daemon % ps aux | grep daemon
ryohunte      52625  86.4  0.0  410604496   432  ??   Rs   11:56pm   4:36.49  ./daemon
ryohunte      52487  84.9  0.0  410603216   352  ??   Rs   11:46pm  13:20.20  ./daemon
ryohunte      48948  21.4  0.0  410604240   320  ??   Rs  Wed11pm  117:44.91  ./daemon
ryohunte      48829  19.9  0.0  410602192   320  ??   Rs  Wed11pm  120:05.91  ./daemon
ryohunte      48299  15.6  0.0  410605264   336  ??   Rs  Wed11pm  158:18.86  ./daemon
ryohunte      47954  15.3  0.0  410605264   320  ??   Rs  Wed10pm  175:39.89  /Users/ryohunte/Desktop/Daemon/daemon
ryohunte      48818  11.5  0.0  410604240   336  ??   Rs  Wed11pm  122:51.01  ./daemon
ryohunte      48649  10.8  0.0  410603216   336  ??   Rs  Wed11pm  134:46.03  ./daemon
ryohunte      48014  10.8  0.0  410604240   336  ??   Rs  Wed10pm  173:35.31  /Users/ryohunte/Desktop/Daemon/daemon
ryohunte      47909  10.4  0.0  410603216   320  ??   Rs  Wed10pm  176:11.63  ./daemon
ryohunte      48793  10.4  0.0  410595024   320  ??   Rs  Wed11pm  123:40.50  ./daemon
ryohunte      49229   9.7  0.0  410604240   336  ??   Rs  12:16am  113:12.80  ./daemon
ryohunte      48893   9.4  0.0  410592976   320  ??   Rs  Wed11pm  118:34.05  ./daemon
ryohunte      48061   8.4  0.0  410604240   320  ??   Rs  Wed10pm  170:15.16  ./daemon
_indowserver  156    2.3  1.0  413239808  83264  ??   Ss   19Apr25  325:40.24  /System/Library/PrivateFrameworks/SkyLight.fram
ework/Resources/WindowServer -daemon
ryohunte      52663   0.0  0.0  410733328   1632 s000  S+   12:01am   0:00.00  grep daemon
_distnote     141    0.0  0.0  426916400   2704  ??   Rs   19Apr25   0:51.31  /usr/sbin/distnoted daemon
root          159    0.0  0.1  426927360   4640  ??   Rs   19Apr25   3:00.19  /usr/sbin/cfprefsd daemon
root          103    0.0  0.0  426912400   1920  ??   Ss   19Apr25   0:00.04  /usr/libexec/IOMFB_bics_daemon
root           97    0.0  0.1  426960000   6576  ??   Ss   19Apr25   0:32.78  /usr/sbin/systemstats --daemon
ryohunte@Ryos-MacBook-Pro Daemon %
```

```

ryohunte@Ryos-MacBook-Pro desktop % cd Daemon
ryohunte@Ryos-MacBook-Pro Daemon % clang daemon.c library.c -o daemon
ryohunte@Ryos-MacBook-Pro Daemon % ./daemon
ryohunte@Ryos-MacBook-Pro Daemon % sudo journalctl -t daemon

```

This shows the daemon is working as no errors come up

Side note: During the beginning of creating the daemon, i came across a lot of errors before it started functioning

```

Last login: Tue Apr 29 16:41:35 on ttys000
ryohunte@Mac ~ % cd Dekstop
cd: no such file or directory: Dekstop
ryohunte@Mac ~ % cd Desktop
ryohunte@Mac Desktop % cd Daemon
ryohunte@Mac Daemon % clang daemon.c -o daemon
ryohunte@Mac Daemon % clang daemon.c library.c -o daemon
library.c:19:59: warning: sizeof on pointer operation will return size of 'char *' instead of 'char[104]' [-Wsizeof-array-decay]
19 | strcpy(addr.sun_path, SOCK_PATH, sizeof(addr.sun_path -1));
   |                                     ^
/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/secure/_string.h:84:33: note: expanded from macro 'strcpy'
84 | __builtin___strcpy_chk (dest, __VA_ARGS__, __darwin_obsz (dest))
   |                               ^
library.c:19:5: error: too many arguments to function call, expected 3, have 4
19 | strcpy(addr.sun_path, SOCK_PATH, sizeof(addr.sun_path -1));
   |     ^~~~~~
/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/secure/_string.h:84:46: note: expanded from macro 'strcpy'
84 | __builtin___strcpy_chk (dest, __VA_ARGS__, __darwin_obsz (dest))
   |                               ^
/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/secure/_common.h:41:31: note: expanded from macro '___darwin_obsz'
41 | #define ___darwin_obsz(object) __builtin_object_size (object, _USE_FORTIFY_LEVEL > 1 ? 1 : 0)
   |                               ^
library.c:27:21: error: incompatible integer to pointer conversion passing 'uint8_t' (aka 'unsigned char') to parameter of type 'const void *' [-Wint-conversion]
27 | write(sock, secret, 16) != 16 ||
   |                 ^~~~~
/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/unistd.h:500:58: note: passing argument to parameter '__buf' here
500 | ssize_t write(int __fd, const void *__libc_sizeof_arg __buf, size_t __nbyte) __DARWIN_ALIAS_C(write);
   |                                     ^
library.c:38:9: error: conflicting types for 'getBlock'
38 | uint8_t getBlock(char *id, uint8_t *secret, uint8_t buffer_length, void *buffer) {
   |         ^~~~~~
./library.h:9:9: note: previous declaration is here
9 | uint8_t getBlock(char *id, uint8_t secret, uint8_t buffer_length, void *buffer);
   |         ^~~~~~
library.c:45:59: warning: sizeof on pointer operation will return size of 'char *' instead of 'char[104]' [-Wsizeof-array-decay]
45 | strcpy(addr.sun_path, SOCK_PATH, sizeof(addr.sun_path -1));
   |                                     ^
/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/secure/_string.h:84:33: note: expanded from macro 'strcpy'
84 | __builtin___strcpy_chk (dest, __VA_ARGS__, __darwin_obsz (dest))
   |                               ^
library.c:45:5: error: too many arguments to function call, expected 3, have 4
45 | strcpy(addr.sun_path, SOCK_PATH, sizeof(addr.sun_path -1));
   |     ^~~~~~
/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/secure/_string.h:84:46: note: expanded from macro 'strcpy'
84 | __builtin___strcpy_chk (dest, __VA_ARGS__, __darwin_obsz (dest))
   |                               ^
/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/secure/_common.h:41:31: note: expanded from macro '___darwin_obsz'
41 | #define ___darwin_obsz(object) __builtin_object_size (object, _USE_FORTIFY_LEVEL > 1 ? 1 : 0)
   |                               ^
2 warnings and 4 errors generated.
ryohunte@Mac Daemon %

```

TC-2 response:

```

ryohunte@Ryos-MacBook-Pro Daemon %
ryohunte@Ryos-MacBook-Pro Daemon % clang client1.c library.c -o client1
ryohunte@Ryos-MacBook-Pro Daemon %

```

For test cases 3 & 4

```

Last login: Fri May 2 01:28:00 on ttys003
ryohunte@Ryos-MacBook-Pro ~ % cd Desktop
ryohunte@Ryos-MacBook-Pro Desktop % cd Daemon
ryohunte@Ryos-MacBook-Pro Daemon % ./client1
storing block failed
ryohunte@Ryos-MacBook-Pro Daemon %

```

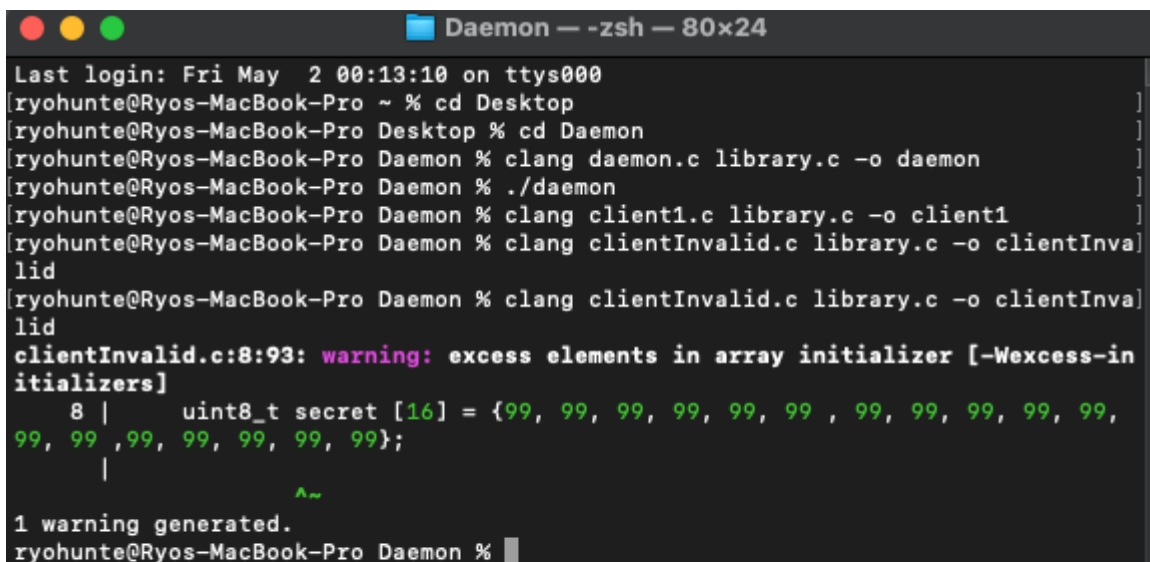
As you can see, the client successfully connects through the unix domain socket, however when trying to store the block, it fails. Whilst the core functions don't seem to work, the library integrates without any problems.

TC- 5 response:

```
ryohunte@Ryos-MacBook-Pro Daemon % ./client2
failed
ryohunte@Ryos-MacBook-Pro Daemon %
```

The retrieval of data fails, but the client program communicates properly through the process.

TC-6:



```
Daemon - zsh - 80x24
Last login: Fri May  2 00:13:10 on ttys000
[ryohunte@Ryos-MacBook-Pro ~ % cd Desktop
[ryohunte@Ryos-MacBook-Pro Desktop % cd Daemon
[ryohunte@Ryos-MacBook-Pro Daemon % clang daemon.c library.c -o daemon
[ryohunte@Ryos-MacBook-Pro Daemon % ./daemon
[ryohunte@Ryos-MacBook-Pro Daemon % clang client1.c library.c -o client1
[ryohunte@Ryos-MacBook-Pro Daemon % clang clientInvalid.c library.c -o clientInval
lid
[ryohunte@Ryos-MacBook-Pro Daemon % clang clientInvalid.c library.c -o clientInval
lid
clientInvalid.c:8:93: warning: excess elements in array initializer [-Wexcess-in
itializers]
    8 |     uint8_t secret [16] = {99, 99, 99, 99, 99, 99, 99, 99, 99, 99, 99,
      |                               ^~
      |                               99, 99, 99, 99, 99, 99};
      |
1 warning generated.
ryohunte@Ryos-MacBook-Pro Daemon %
```

Before executing, an error occurs meaning the security of the server is good as it will not accept a buffer overflow preventing it from DoS attacks.

In conclusion, although the system is not fully functional, it demonstrates how the daemon, library and client programs would communicate with each other.

Real world research vs my project:

After thoroughly doing some research, a real life data sharing system that I found interesting is the Redis database. Redis is an open source, key value, NOSQL database often used for fast caching and a database. The system essentially makes an application closer to external data sources with faster read and write speeds by storing all data in memory rather than a disk or an SSD. This means that the data is physically closer to the application for low latency.

Comparison table:

Feature :	Redis:	My data sharing system:
Data model	Key, value store with different data types (lists, hashes, bit arrays, binary safe data)	Structured blocks (ID,data, permissions)
Communication	TCP socket, REDis serialisation protocol (port 6379)	Unix domain socket and custom C functions
Authentication	Optional password and client ACL's	Block level secrets and role based permission
API	Language bindings	C library with sendNewBlock() and getBlock() functions
Robustness	Highly tested and supports failover	Local memory only with no crash recovery
Scalability	Supports multithreading and multi client	Single machine communication only and single client at a time
Security	Supports TLS, authentication and user based ACL'S	Per block secrets
Functionality	caching , pub/sub messaging. Persistence and replication	Create and read secure data blocks

Conclusion from Redis:

Redis offers several features that highlight areas for potential improvement for my system. The first being data persistence; Redis supports this through snapshotting(RDB) and append only files (AOF) which allows for data recovery after crashes. This is something my system currently lacks as all data is stored in volatile memory. To improve robustness, the new system could serialise the blocks to disk and reload them on startup.

In terms of authentication and access control, Redis uses password based authentication and user based ACL'S. My system currently relies on block based secrets and role based permissions however, to enhance security, the system could implement UID based authentication.

As mentioned earlier, the current system is not that portable as it utilises C structs over unix sockets, on the other hand, Redis is a well defined text based protocol which makes it highly portable and language agnostic. If ported to another language, the system could benefit from a defined message protocol for compatibility.

In terms of scalability, my current system is limited to one client and a single machine and is built for simplicity, compared to Redis which supports thousands of concurrent clients. My current program could adopt multithreading or a queue for better performance under load.

Usability is an aspect that my program really lacks, currently usability is very poor. The implementation of a simple CLI would make it easier to interact with and enhance the developer experience. Redis offers the Redis-CLI and GUI tools to tackle this aspect.

While the domain sockets are generally safe, it does not encrypt data whereas Redis utilises TLS.

Introducing end to end encryption such as OpenSSL or libsodium would protect sensitive fields such as the secret and block data. This would prevent attackers from gaining access to the communication channel or memory.

In conclusion, Redis is a mature data sharing system designed for speed and reliability. Whilst my project is more focused on local machine and single client communication, Redis offers a lot of great features that I can learn from to implement in this project or future data sharing systems. Ultimately Redis offers a scalable, recoverable and flexible system.

Final conclusion:

This project presents the foundational design and partial implementation of a secure interprocess communication system using unix domain sockets and POSIX C. While the system is not fully functioning, the core components such as the daemon, C library, clients and data structuring logic have all been implemented, demonstrating the intended architecture and data flow. Overall this project enhanced my understanding of data sharing systems and low level systems programming.

Appendix:

Daemon.c

```
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <syslog.h>
#include <arpa/inet.h>

#define MAX_DTA_SIZE 16 //defining maximum size to prevent memory overflow
#define SOCKET_PATH "/tmp/daemon.sock" //creating socket file path
#define MAX_ID_SIZE 64
#define MAX_BLOCKS 100
#define PERMISSION_READ 0x1
#define PERMISSION_WRITE 0x2

typedef struct { //structuring the management of data as a data unit
    everytime it is called
    char id[MAX_ID_SIZE];
    uint8_t secret [16]; //16 byte password to protect it
    uint32_t length_data;
    void*data;//pointer to where data is stored
    uint8_t permission;
} Block;

Block *blocks[MAX_BLOCKS];
int blocks_count = 0;

//store data function

void storeblock(const char id[], const uint8_t secret[], const uint32_t
length_data, const void* data, uint8_t permission) {
    //error handling for data management
    if (blocks_count >= MAX_BLOCKS) {
        syslog(LOG_ERR, "Too many blocks!");
        return;
    }

    Block *new_b = (Block *)malloc(sizeof(Block)); //allocating memory to the
    if (!new_b) {
        syslog(LOG_ERR, "Out of memory!");
        return;
    }
}
```

```

memcpy(new_b->id, id, MAX_ID_SIZE);
memcpy(new_b->secret, secret, 16);
new_b->length_data = length_data;
new_b->data = malloc(length_data);
new_b->permission = permission;
if (!new_b->data) {
    free(new_b);
    syslog(LOG_ERR, "Out of memory!");
    return;
}
memcpy(new_b->data, data, length_data);
blocks[blocks_count++] = new_b;
syslog(LOG_INFO, "Created new block %s", new_b->id);
}

void retrieveBlock(int client_fd) {
    char id[MAX_ID_SIZE];
    uint8_t secret [16];

    if (read(client_fd, id, sizeof(id)) != sizeof(id)) return;
    if (read(client_fd, secret , 16) != 16) return;

    for (int i = 0; i < blocks_count; i++) {
        if (strncmp(blocks[i]->id, id, MAX_ID_SIZE) == 0&&
            memcmp(blocks[i]->secret, secret, 16) == 0) {
            if (!(blocks[i]->permission & PERMISSION_READ)){
                syslog(LOG_WARNING, "Access denied for block ID:  %s",id);
                return;
            }
            uint32_t data_length = blocks[i]->length_data;
            if (write(client_fd, &data_length, sizeof(data_length))!=
sizeof(data_length)) {
                syslog(LOG_ERR, "failed to send data length");
                return;
            }
            write(client_fd, &blocks[i]->data,sizeof(blocks[i]->data));
            write(client_fd, blocks[i]->data, blocks[i]->length_data);
            syslog(LOG_INFO, "Read block ID: %s", blocks[i]->id);
            return;
        }
    }
    syslog(LOG_WARNING, "Could not read block ID: %s", id);
}

//handle process request functions
void requests(int client_fd){
    char command;
    if (read(client_fd, &command, 1 )!=1) return;

```

```

if (command == 'S') {
    char id[MAX_ID_SIZE];
    uint8_t secret[16];
    uint32_t length_data;
    char data[MAX_DTA_SIZE];
    if(read(client_fd, &id, sizeof(id)) != sizeof(id)) return;
    if(read(client_fd, secret, 16) != 16) return;
    //read data length
    if(read(client_fd, &length_data, sizeof(length_data)) !=
sizeof(length_data)) return;
    if(length_data > MAX_DTA_SIZE) return;

    if(read(client_fd, data, sizeof(data)) != sizeof(data)) return;

    storeblock(id, secret, length_data, data, PERMISSION_READ |
PERMISSION_WRITE);

}else if (command == 'G') {
    retrieveBlock(client_fd);
}else {
    syslog(LOG_ERR, "Invalid command");
}
}

```

```

int main(void){

    openlog("daemon", LOG_PID | LOG_CONS, LOG_DAEMON);
    syslog(LOG_INFO, "Starting daemon");

    pid_t pid = fork();
    if (pid < 0) {
        exit(EXIT_SUCCESS);
    }
    if (pid > 0) {
        exit(EXIT_SUCCESS);
    }
    //resetting the file mode
    umask(0);
    //create new session
    pid_t sid = setsid();
    if (sid < 0) {
        exit(EXIT_SUCCESS);
    }
    //Next, make the current directory
    if((chdir("/") < 0)) {
        syslog(LOG_ERR, "%s\n", "chdir");
        exit(EXIT_FAILURE);
    }
    //make directory
    if((chdir("/") < 0)) {
        syslog(LOG_ERR, "%s\n", "chdir");
        exit(EXIT_FAILURE);
    }
}

```

```

    }
    /* Close stdin, etc. */
    close(STDIN_FILENO);
    close(STDOUT_FILENO);
    close(STDERR_FILENO);

    //setting up the unix domain socket for IPC communication
    int server_fd, client_fd;
    server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, "/tmp/daemon.sock", sizeof(addr.sun_path)-1);
    unlink(addr.sun_path);

    //binding server error handling
    if(bind(server_fd, (struct sockaddr *) &addr, sizeof(addr)) < 0) {
        syslog(LOG_ERR, "Binding failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    syslog(LOG_INFO, "Daemon is running at %s", SOCKET_PATH);

    while(1) {
        //used to connect the clients
        int client_fd = accept(server_fd, NULL, NULL);
        if (client_fd < 0) continue;
        requests(client_fd);
        close(client_fd);
    }
    close(server_fd);
    unlink(SOCKET_PATH);
    syslog(LOG_INFO, "Done");
    closelog();
    return 0;
}

```

Library.c:

```

#include "library.h"
#include <stdio.h>
#include <string.h>
#include <sys/un.h>
#include <unistd.h>
#include <sys/un.h>
#include <stdint.h>
#include <sys/socket.h>

#define SOCK_PATH "/tmp/daemon.sock"

```

```

uint8_t sendNewBlock(char *id, uint8_t *secret, uint32_t data_length, void
*data) {
    int sock = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sock < 0) return 1;

    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, SOCK_PATH, sizeof(addr.sun_path) -1 );

    if (connect(sock, (struct sockaddr *) &addr, sizeof(addr)) < 0) {
        close(sock);
        return 1;
    }

    if (write(sock, "S", 1) != 1 ||
        write(sock, id, 64) != 64 ||
        write(sock, secret, 16) != 16 ||
        write(sock, &data_length, sizeof(data_length)) != sizeof(data_length) ||
        write(sock, data, data_length) != data_length){
        close(sock);
        return 1;
    }

    close(sock);
    return 0;
}

```

```

uint8_t getBlock(char *id, uint8_t *secret, uint8_t buffer_length, void
*buffer) {
    int sock = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sock < 0) return 1;

    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strcpy(addr.sun_path, SOCK_PATH);

    if (connect(sock, (struct sockaddr *) &addr, sizeof(addr)) < 0) {
        close(sock);
        return 1;
    }

    if (write(sock, "G", 1) != 1 ||
        write(sock, id, 64) != 64 ||
        write(sock, secret, 16) != 16) {
        close(sock);
        return 1;
    }

    uint32_t data_length;
    if (read(sock, &data_length, sizeof(data_length)) !=
sizeof(data_length)) {
        close(sock);
    }
}

```

```

        return 1;
    }
    if (data_length > buffer_length) {
        close(sock);
        return 1;
    }
    if (read(sock, buffer, data_length) != data_length) {
        close(sock);
        return 1;
    }
    close(sock);
    return 0;
}

```

library.h:

```

#ifndef LIBRARY_H
#define LIBRARY_H

#include <stdint.h>

#define SOCKET_PATH "/tmp/socket"

//id = unique string identifier for block
//secret = 16 byte authentication string
//data_length = length of data in bytes
//data = pointer to blocks data
//buffer_length = how much data was copied from memory
//buffer = pointer to pre allocated memory

uint8_t sendNewBlock(char *id, uint8_t *secret, uint32_t data_length, void
*data);
uint8_t getBlock(char *id, uint8_t *secret, uint8_t buffer_length, void
*buffer);

#endif //LIBRARY_H

```

Client1

```

#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include "library.h"

int main() {
    char id[64] = {0};
    strncpy(id, "Block1", sizeof(id)-1);
    uint8_t secret [16] = {0};
    char data[] = "Hello";
    uint32_t data_length = strlen(data) + 1;
}

```

```

uint8_t clientdata = sendNewBlock(id, secret, data_length, data);

if (clientdata == 0) {
printf("stored block \n");
}
else {
printf("storing block failed\n");
}

return 0;
}

```

Client2.c:

```

#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include "library.h"

int main() {
    char id[] = "block1";
    uint8_t secret [16] = {0};
    char buffer [128];

    uint8_t client2data = getBlock(id, secret, sizeof(buffer), buffer);

    if (getBlock(id, secret, sizeof(buffer), buffer) == 0) {
printf("success\n");
    }
    else {
printf("failed\n");
    }

    return 0;
}

```

clientInvalid.c

```

#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include "library.h"

int main() {
    char id[] = "block1";
    uint8_t secret [16] = {99, 99, 99, 99, 99, 99 , 99, 99,
99,99,99,99,99,99,99,99,99};
    char buffer [128];

    uint8_t client2data = getBlock(id, secret, sizeof(buffer), buffer);

```

```
    if (getBlock(id, secret, sizeof(buffer), buffer) == 0) {
        printf("success\n");
    }
    else {
        printf("failed\n");
    }

    return 0;

}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.29)
project(Daemon C)

set(CMAKE_C_STANDARD 11)

add_executable(Daemon daemon.c)

add_library(library STATIC library.c
              library.c)

add_executable(client1 client1.c
               client1.c)
target_link_libraries(client1 library)

add_executable(client2 client2.c
               client2.c
               client2.c)
target_link_libraries(client2 library)

add_executable(clientInvalid clientInvalid.c
               clientInvalid.c)
target_link_libraries(clientInvalid library)
```

